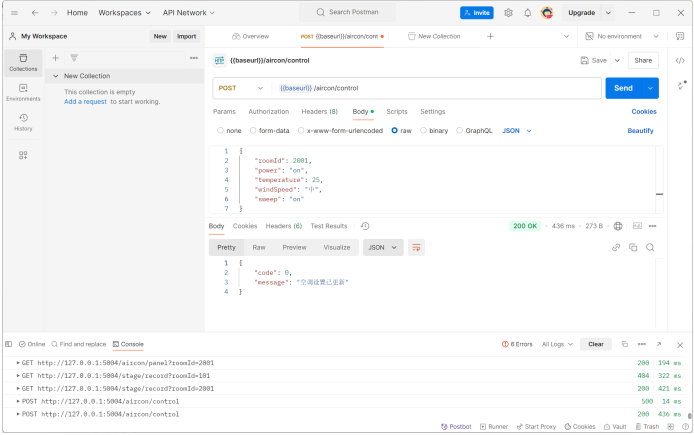
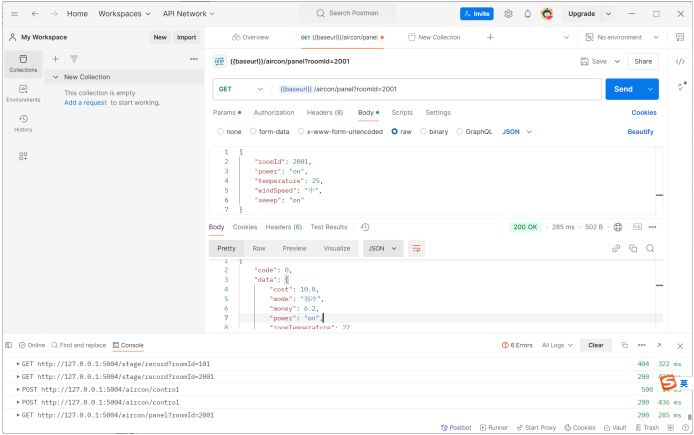


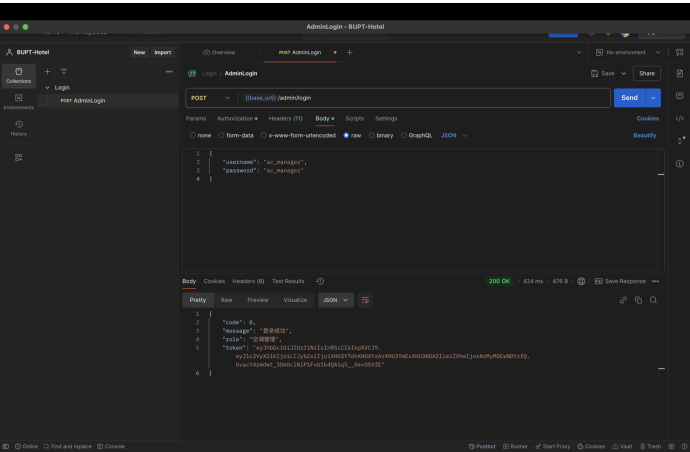
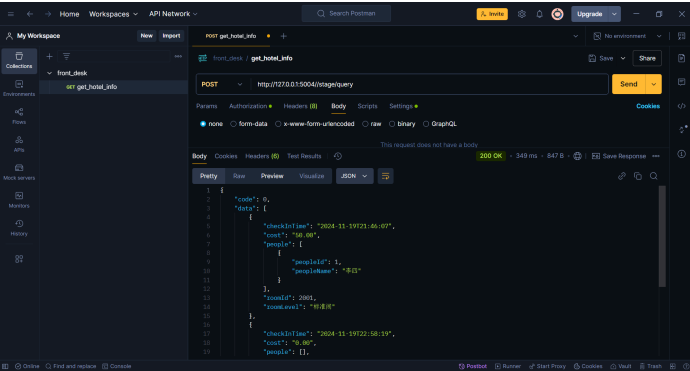
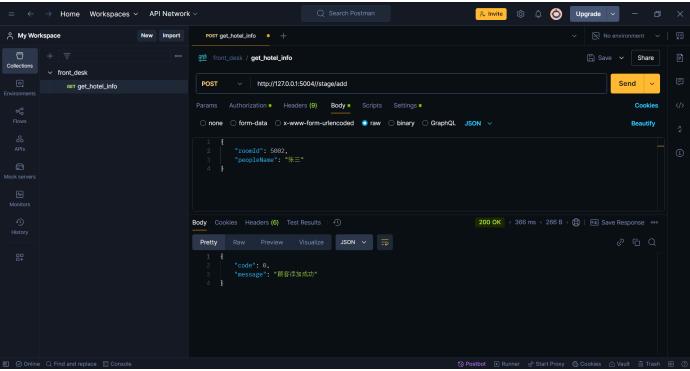
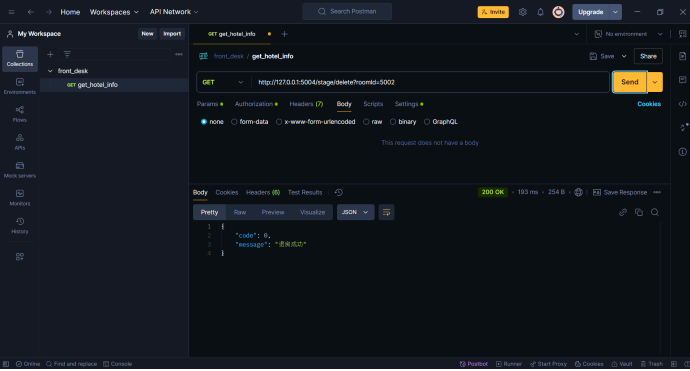
BUPT-Hotel后端架构文档

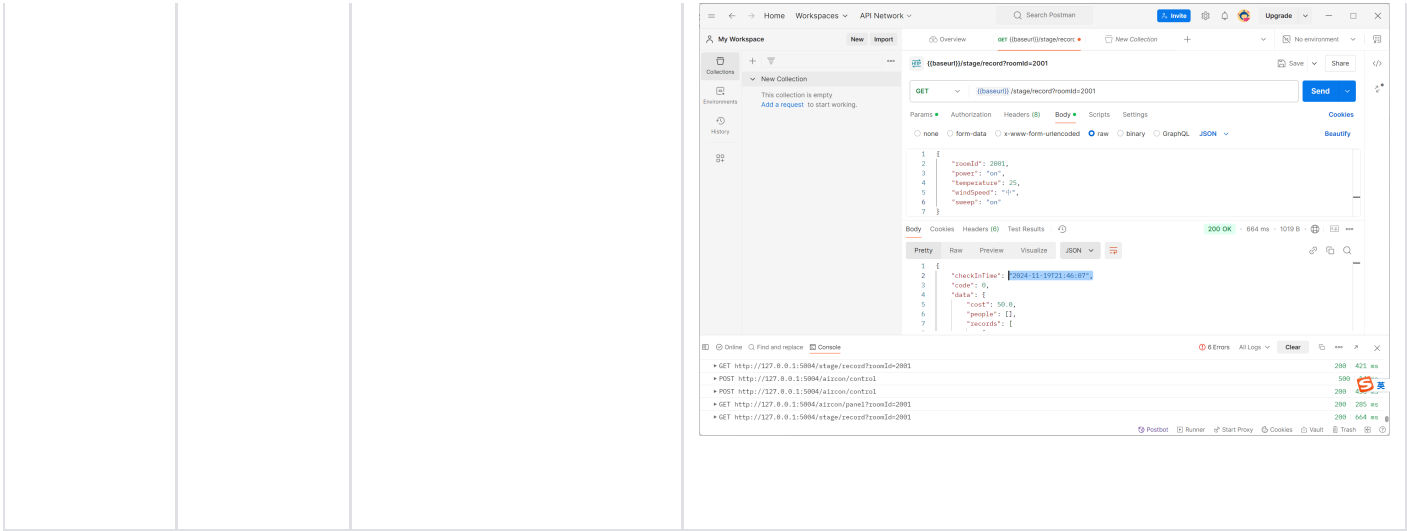
🕒 2024年11月25日 17:30

1. 后端分工

接口文档详见：[📄 软件工程联合验收](#)

接口名称	接口负责人	接口开发是否已完成	接口调试截图
空调调整接口	池瀚	☒ 接口开发已完成	
获取该房间当前情况接口	池瀚	☒ 接口开发已完成	
管理员登录接口	王殿云	☒ 接口开发已完成	

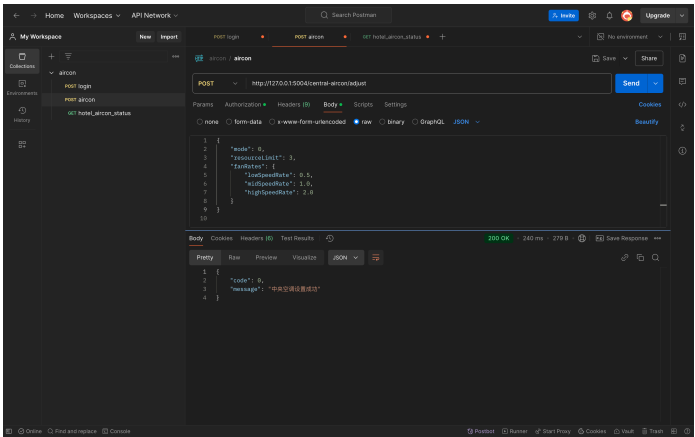
			
获取整个酒店入住情况接口	王艺栋	☑ 接口开发已完成	
办理入住接口	王艺栋	☑ 接口开发已完成	
办理退房接口	王艺栋	☑ 接口开发已完成	
开具详单接口	池瀚	☑ 接口开发已完成	



中央空调设置

王博远

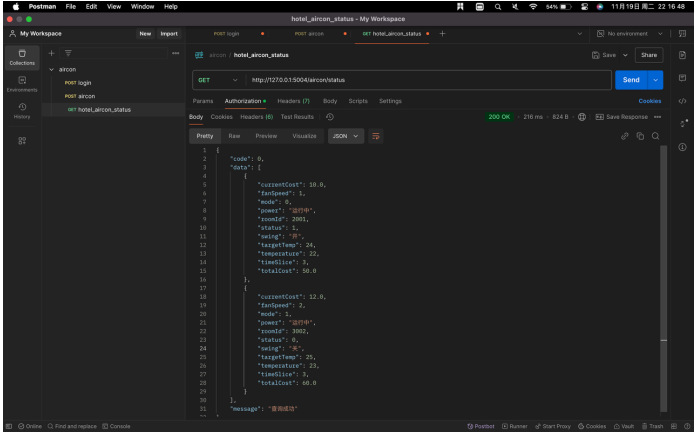
☑ 接口开发已完成



获取整个酒店空调情况

王博远

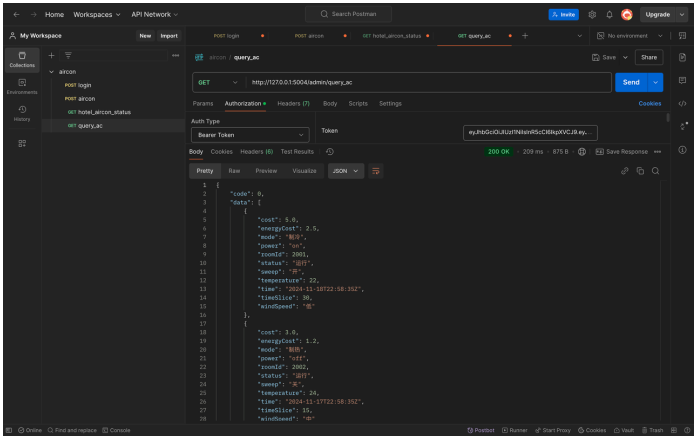
☑ 接口开发已完成

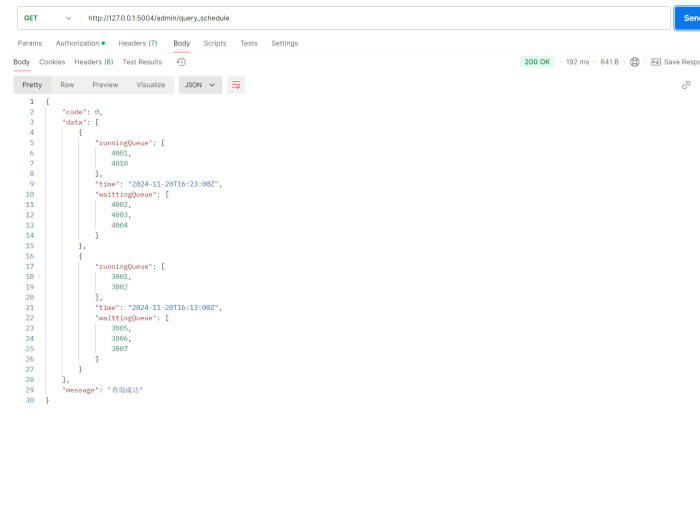
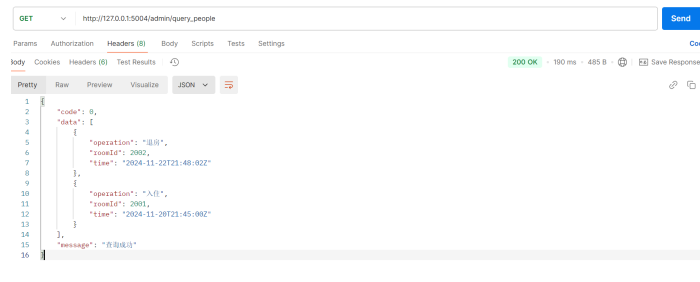
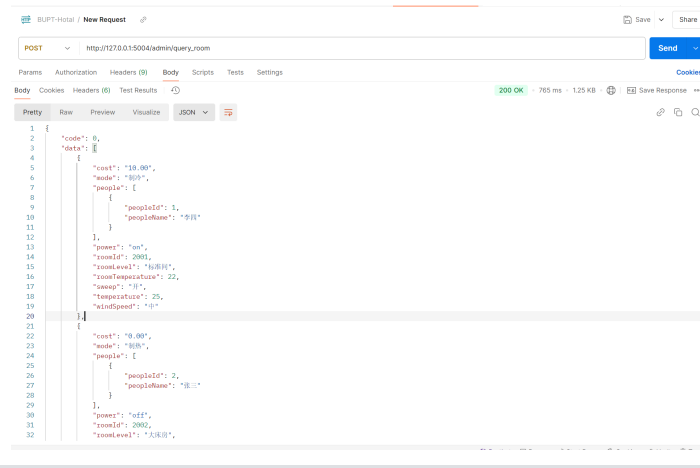


获取近一周空调操作记录

王博远

☑ 接口开发已完成



获取近一周调度记录	王博涵	☑ 接口开发已完成	
获取近一周客流记录	王博涵	☑ 接口开发已完成	
获取各房间信息接口	王博涵	☑ 接口开发已完成	

2. 项目结构

项目采用 Flask 框架，遵循 MVC (Model-View-Controller) 架构设计模式，并使用服务层处理业务逻辑。项目结构如下：

代码块	
1	backend/
2	├─ app.py
3	├─ config/
4	│ └─ config.yaml
5	├─ controllers/
6	│ └─ login_controller.py
7	├─ mappers/
8	│ └─ admin_login_mapper.py
9	└─ services/
	# Flask 应用主入口
	# 配置文件目录
	# 主配置文件
	# 控制器层，处理路由和请求
	# 登录相关控制器
	# 数据访问层
	# 管理员登录数据访问
	# 业务逻辑层

```
10 |   └─ admin_login_service.py # 管理员登录业务逻辑
11 |   └─ sql/                    # SQL脚本目录
12 |   └─ hotel_air_conditioning.sql # 数据库初始化脚本
13 |   └─ utils/                  # 工具类
14 |       └─ config_utils.py     # 配置工具
15 |       └─ database_utils.py   # 数据库工具
16 |       └─ jwt_utils.py        # JWT工具
17 |       └─ password_utils.py   # 密码处理工具
```

3. 基础设施

3.1 Database 数据库工具

`utils/database_utils.py` 提供了数据库连接池管理：

3.1.1 基本使用方式

所有数据库操作都遵循以下格式：

代码块

```
1  try:
2      with current_app.db as cursor:
3          cursor.execute(你的SQL语句, 参数元组)
4          # 根据需要获取结果
5  except Exception as e:
6      raise Exception(f"操作失败: {e}")
```

3.1.2 常见操作参考

1. 查询操作

查询一条记录：

代码块

```
1  try:
2      with current_app.db as cursor:
3          cursor.execute("SELECT * FROM 表名 WHERE 条件 = %s", (参数,))
4          result = cursor.fetchone()
5  except Exception as e:
6      raise Exception(f"操作失败: {e}")
```

查询多条记录：

代码块

```
1 try:
2     with current_app.db as cursor:
3         cursor.execute("SELECT * FROM 表名 WHERE 条件 = %s", (参数,))
4         results = cursor.fetchall()
5 except Exception as e:
6     raise Exception(f"操作失败: {e}")
```

2. 插入操作

代码块

```
1 try:
2     with current_app.db as cursor:
3         cursor.execute("INSERT INTO 表名 (字段1, 字段2) VALUES (%s, %s)", (值1,
4                                     值2))
5         new_id = cursor.lastrowid # 获取新插入的ID
6 except Exception as e:
7     raise Exception(f"操作失败: {e}")
```

3. 更新操作

代码块

```
1 try:
2     with current_app.db as cursor:
3         cursor.execute("UPDATE 表名 SET 字段 = %s WHERE 条件 = %s", (新值, 条件
4                                     值))
5         is_success = cursor.rowcount > 0 # 检查是否更新成功
6 except Exception as e:
7     raise Exception(f"操作失败: {e}")
```

4. 删除操作

代码块

```
1 try:
2     with current_app.db as cursor:
3         cursor.execute("DELETE FROM 表名 WHERE 条件 = %s", (条件值,))
4         is_success = cursor.rowcount > 0 # 检查是否删除成功
5 except Exception as e:
6     raise Exception(f"操作失败: {e}")
```

5. 提示

- a. 记得写 try-except 进行错误处理

- b. SQL语句中用 %s 作为参数占位符
- c. 参数要用元组 () 传入，即使只有一个参数也要加逗号
- d. 查询结果的下标从0开始

3.2 JWT 认证

`utils/jwt_utils.py` 提供JWT令牌管理：

代码块

```
1  # 创建token
2  jwt_manager = get_jwt_manager()
3  token = jwt_manager.create_token(user_id, role)
4
5  # 验证token
6  token_info = jwt_manager.verify_token(token)
7  if token_info:
8      user_id = token_info['user_id']
9      role = token_info['role']
```

功能：

- 支持角色信息编码
- 自动处理令牌过期
- 可配置加密算法和密钥

3.3 密码工具

`utils/password_utils.py` 提供密码加密和验证：

代码块

```
1  # 密码加密
2  hashed = hash_password(password)
3
4  # 密码验证
5  is_valid = verify_password(password, hashed)
```

特点：

- 使用bcrypt加密算法
- 自动处理盐值生成
- 支持配置加密强度

3.4 配置管理

`utils/config_utils.py` 提供配置文件加载和访问：

代码块

```
1  # 加载配置 app.py
2  config = Config('./config/config.yaml')
3  config.to_flask_config(app)
4
5  # 访问配置
6  import current_app
7  value = current_app.config['key']['subkey']
```

4. 核心模块说明

4.1 Controllers 控制器层

位于 `controllers/` 目录，负责处理HTTP请求、参数验证和返回响应。

示例（登录控制器）：

代码块

```
1  @app.route('/admin/login', methods=['POST'])
2
3  def admin_login_controller():
4      data = request.get_json()
5      # 参数验证和业务处理
6      result = admin_login(data['username'], data['password'])
7      return jsonify(result)
```

4.2 Mappers 数据访问层

位于 `mappers/` 目录，负责数据库操作，提供数据访问接口。

示例（查询管理员）：

代码块

```
1  def get_admin_by_username(username):
2      with current_app.db as cursor:
3          query = """
4              SELECT AdminID, Username, PasswordHash, Role
5              FROM Administrators
6              WHERE Username = %s
```



```
7         """
8         cursor.execute(query, (username,))
9         return cursor.fetchone()
```

4.3 Services 服务层

位于 `services/` 目录，处理业务逻辑，调用mapper层进行数据操作。

代码块

```
1 def admin_login(username, password):
2     admin = get_admin_by_username(username)
3     if verify_password(password, admin['password_hash']):
4         return create_login_response(admin)
5     return None
```